AI Agent Code Coverage: Ablation Study

Prompt, Knowledge, Model, and Framework Effects on AI-Generated Tests

Mark Pollack

March 2026 — DevNexus

Abstract

We evaluate the effect of prompt engineering, domain knowledge injection, structured execution, model selection, and agent framework on AI-generated test quality for Spring Boot projects. Eleven variants are tested across two agent frameworks (Claude Code CLI and a Spring AI agent loop), three models (Claude Sonnet 4.6, Claude Haiku 4.5, Qwen3-Coder 30B), and progressive knowledge treatments including a Stable Analysis Envelope (SAE) — a pre-computed project report with copy-paste import blocks. Each variant is evaluated by a four-tier automated judge cascade measuring build success, coverage preservation, practice adherence across six criteria, and structural similarity to expert-written reference tests. On simple projects, a hardened prompt is the dominant lever (+29% quality, same model). Knowledge amplifies capable models (Sonnet + SAE: T3=0.93) but cannot lift weak ones (Haiku + SAE: T3=0.35). Below a model capability floor, no amount of tooling or knowledge helps. Structured two-phase execution adds cost on trivial targets but achieves the highest quality on complex multi-layer applications (Pet Clinic plan/act: T3=0.95). The model determines *whether it works*; the framework determines *what it costs*.

Contents

1 Experimental Design **3**
- 1.1 Scoring: Four Dimensions . . . 3
  - 1.1.1 Line Coverage (JaCoCo) . . . 3
  - 1.1.2 T3 Practice Adherence (LLM Judge) . . . 3
  - 1.1.3 Golden Test Similarity (AST Comparison) . . . 4
  - 1.1.4 Efficiency (Agent Behavior) . . . 4
- 1.2 Judge Cascade . . . 5
- 1.3 Agent Execution and Capture . . . 5

2 Results at a Glance **6**

3 Key Findings **6**
- 3.1 Finding 1: Prompt Engineering Is the Biggest Lever . . . 6
- 3.2 Finding 2: Model Is the Quality Gate, Framework Is the Cost Gate . . . 7
- 3.3 Finding 3: There Is a Model Floor . . . 7
- 3.4 Finding 4: Knowledge Amplifies Capability . . . 8
- 3.5 Finding 5: Local Models Are Not Ready . . . 8
- 3.6 Finding 6: Partial Knowledge Costs More Than Full or None . . . 9
- 3.7 Finding 7: Two-Phase Costs 2× for No Gain . . . 9

4 Agent Behavior **9**
- 4.1 Agent Internal Reasoning (Thinking Blocks) . . . 9
- 4.2 Token Consumption . . . 12
- 4.3 Tool Usage Patterns . . . 13

4.4 Execution Timelines 14

5 Per-Item Breakdown **16**

6 Efficiency **17**

7 Discussion **17**
7.1 Limitations 18

8 Conclusion **18**

1 Experimental Design

Task: Given a Spring Boot project with production code and *no tests*, generate a JUnit test suite targeting ≥80% line coverage.

Dataset: 5 Spring Getting Started guides (trivial, 2–3 classes) + Spring Pet Clinic (complex, 15+ classes, 4 layers).

Notation: `Agent:Model` — e.g., `CC:Sonnet` = Claude Code CLI running Claude Sonnet 4.6; `Loopy:Haiku` = Loopy framework (Spring AI agent) running Claude Haiku 4.5.

Variants tested — two axes of variation:

Agent:Model	Prompt	Knowledge	What it tests
CC:Sonnet (naive)	v0-naive	none	Baseline — minimal instructions
CC:Sonnet (hardened)	v1-hardened	none	Does prompt quality matter?
CC:Sonnet (+3 KB)	v2-with-kb	3 files	Does curated knowledge help?
CC:Sonnet (+full KB)	v2-with-kb	full KB	Does more knowledge help more?
CC:Sonnet (two-phase)	v3-explore+act	full KB	Does structured execution help?
CC:Sonnet (+SAE)	v1-hardened+SAE	none	Does pre-computed analysis help Sonnet?
CC:Haiku	v1-hardened	none	Can a cheap model match Sonnet?
CC:Haiku (+SAE)	v1-hardened+SAE	none	Does pre-computed analysis help Haiku?
Loopy:Haiku	v1-hardened	none	Does the framework matter?
Loopy:Sonnet	v1-hardened	none	Does Sonnet fix Loopy's quality gap?
Loopy:Qwen3	v1-hardened	none	Can a free local model do the job?

Scoring: 4-tier automated judge cascade — build success (T0), coverage preservation (T1), coverage improvement + golden test similarity (T2), LLM practice adherence across 6 criteria (T3).

1.1 Scoring: Four Dimensions

Each variant–item result is scored on four independent dimensions. Scores are **not combined** into a single number — they measure different aspects of test quality.

1.1.1 Line Coverage (JaCoCo)

Standard JaCoCo line coverage percentage. Measured deterministically by running `mvn clean test jacoco:report` on the agent's workspace after completion. Reports both baseline (always 0% since no tests exist initially) and final coverage.

Interpretation: Higher is better, but coverage alone does not measure test quality — a test that calls every method without assertions achieves high coverage with no value.

1.1.2 T3 Practice Adherence (LLM Judge)

An LLM-based judge (Claude Sonnet 4.6) evaluates the generated test suite against six criteria derived from Spring testing best practices:

1. **Test slice selection** — Does the test use the appropriate Spring test annotation (`@WebMvcTest`, `@DataJpaTest`, etc.) rather than `@SpringBootTest` everywhere?

2. **Assertion quality** — Are assertions specific and domain-aware (e.g., checking actual field values) rather than trivial (`assertNotNull`)?

3. **Error and edge case coverage** — Does the suite test validation failures, missing entities, boundary conditions?

4. **Domain-specific patterns** — Are Spring-specific testing patterns used correctly (MockMvc for controllers, TestEntityManager for JPA)?
5. **Coverage target selection** — Does the suite focus on behaviorally significant code rather than trivial getters?
6. **Version-aware patterns** — Does the suite use APIs appropriate for the detected Spring Boot version (Boot 4.x vs 3.x)?

Each criterion is scored 0.0–1.0 by the judge with supporting evidence. **T3** is the mean across all six criteria. The rubric is identical for all variants — knowledge files inform the agent's test authorship, not the judge's evaluation.

Interpretation: T3 measures *how well* the tests follow expert practices, independent of whether they compile or achieve coverage.

1.1.3 Golden Test Similarity (AST Comparison)

A deterministic comparison of the agent's test suite against expert-written reference ("golden") tests using JavaParser AST analysis. Five weighted dimensions:

- **Test method coverage** (weight 0.30) — ratio of reference test methods that have a corresponding agent test
- **Annotation alignment** (weight 0.20) — overlap in Spring test annotations used
- **Assertion style** (weight 0.20) — ratio of assertion count (agent vs reference)
- **Import alignment** (weight 0.15) — overlap in test imports (proxy for testing approach)
- **Injection pattern** (weight 0.15) — overlap in injected dependencies (`@MockitoBean`, `@Autowired`)

Interpretation: Golden similarity measures structural resemblance to expert tests. A score of 1.0 means the agent produced tests structurally identical to the reference. This is *not* an LLM judge — it is fully deterministic.

1.1.4 Efficiency (Agent Behavior)

Measures how efficiently the agent used its resources. Four weighted sub-metrics:

- **Build errors** (weight 0.35) — 1.0 − (failed builds/total builds). An agent that compiles correctly on every attempt scores 1.0.
- **Cost efficiency** (weight 0.20) — normalized inverse of total cost. Lower cost = higher score.
- **Recovery cycles** (weight 0.20) — 1.0 − (recovery iterations/total iterations). An agent that never needs to fix a broken build scores 1.0.
- **Tool utilization** (weight 0.25) — ratio of productive tool calls to total tool calls.

Interpretation: Efficiency measures the agent's operational quality — did it work cleanly, or did it thrash through compile-fix-compile cycles? Two agents can produce identical tests but differ dramatically in efficiency.

1.2 Judge Cascade

Judges are organized in a four-tier cascade. Lower tiers are binary gate checks; higher tiers produce continuous scores. If a lower tier rejects (e.g., build fails), higher tiers do not execute.

Tier 0: **BuildSuccessJudge** — does the project compile and all tests pass? (binary gate)

Tier 1: **CoveragePreservationJudge** — no regression from baseline coverage. (binary gate)

Tier 2: **CoverageImprovementJudge** + **GoldenTestComparisonJudge** — how much did coverage improve? How similar are the tests to expert reference tests? (continuous scores)

Tier 3: **TestQualityJudge** — LLM-based practice adherence across 6 criteria. (continuous score)

1.3 Agent Execution and Capture

Each agent invocation is captured via session log parsing (`SessionLogParser` from the agent-journal library). Both `AgentClient` and `ClaudeSyncClient` produce structured session logs that are parsed into a common `PhaseCapture` record. The captured exhaust includes:

- **Tool calls**: every file read, write, edit, glob, grep, and bash command
- **Token usage**: input tokens, output tokens, thinking tokens (extended thinking)
- **Turn count**: number of agent–environment round trips
- **Cost**: USD cost per invocation (API pricing)
- **Duration**: wall-clock time from prompt to final response
- **Phase count**: 1 for single-phase variants, 2 for two-phase (explore + act)

For two-phase variants, both phases run in the same `ClaudeSyncClient` session — `client.connect(explor` followed by `client.query(actPrompt)`. The explore phase's full context (tool results, reasoning) is preserved in the session for the act phase. Each phase's exhaust is captured separately via `SessionLogParser.parse()`.

Table 1: Variant comparison (N=1 per cell, sorted by variant progression)

Agent:Model	N	Pass%	Cov%	T3	Eff	Total	Per Item
CC:Sonnet (naive)	5	100%	95.6	0.62	0.88	$4.57	$0.91
CC:Sonnet (hardened)	5	100%	85.7	0.80	0.94	$4.17	$0.83
CC:Sonnet (+3 KB)	5	100%	95.6	0.70	0.84	$4.98	$1.00
CC:Sonnet (+full KB)	5	100%	84.9	0.75	0.85	$5.13	$1.03
CC:Sonnet (two-phase)	6	100%	94.3	0.72	0.65	$16.48	$2.75
CC:Sonnet (forge)	1	100%	95.3	0.95	0.17	$8.55	$8.55
CC:Sonnet (+SAE)	1	100%	71.4	0.93	0.86	$0.92	$0.92
CC:Haiku	6	17%	78.5	0.44	0.77	$2.88	$0.48
CC:Haiku (+SAE)	1	0%	71.4	0.35	0.41	$0.48	$0.48
Loopy:Sonnet	1	100%	94.6	0.73	0.86	$2.72	$2.72
Loopy:Haiku	6	33%	71.4	0.49	0.86	$15.30	$2.55
Loopy:Qwen3	1	0%	0	—	0.73	$5.01	$5.01

Table 2: Judge cascade results per variant (tier 0–3 + golden)

Judge	CC:Sonnet (naive)	CC:Sonnet (hardened)	CC:Sonnet (+3 KB)	CC:Sonnet (+full KB)
Build Success	100%	100%	100%	100%
Coverage Preservation	100%	100%	100%	100%
Coverage Improvement	100%	100%	100%	100%
Practice Adherence	0.62	0.80	0.70	0.75
Golden Similarity	nan	nan	nan	0.68

2 Results at a Glance

3 Key Findings

3.1 Finding 1: Prompt Engineering Is the Biggest Lever

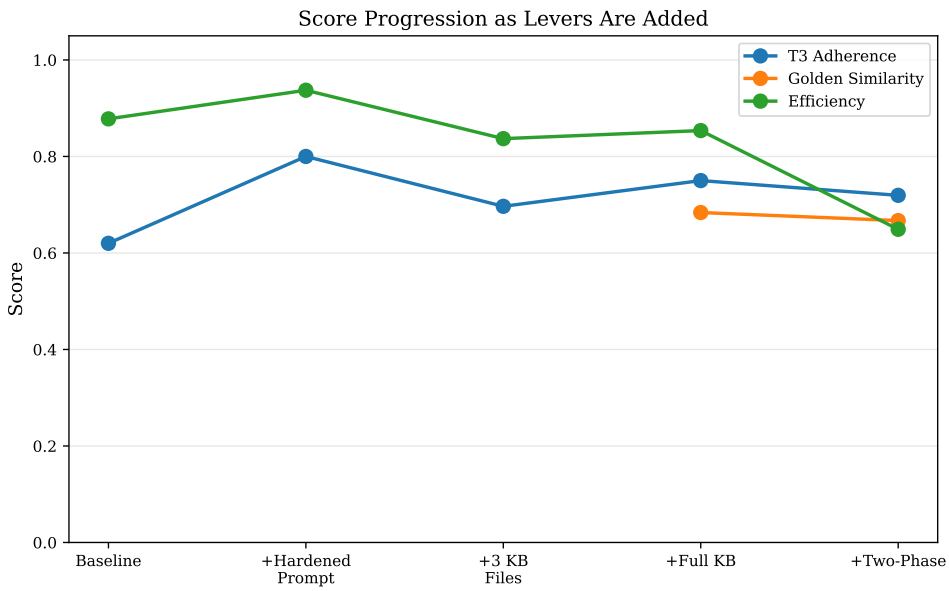

Figure 1: Score progression as levers are added. The jump from naive to hardened prompt is the dominant improvement.

Table 3: Cost and token breakdown per variant

Agent:Model	Total	Per Item	In Tok	Out Tok	Think Tok	Duration
CC:Sonnet (naive)	$4.57	$0.91	109	55,430	10,160	1513s
CC:Sonnet (hardened)	$4.17	$0.83	137	49,343	7,791	1481s
CC:Sonnet (+3 KB)	$4.98	$1.00	137	59,472	11,328	1576s
CC:Sonnet (+full KB)	$5.13	$1.03	1,119	56,017	9,924	1671s
CC:Sonnet (two-phase)	$16.48	$2.75	6,953	158,614	29,197	3046s
CC:Sonnet (forge)	$8.55	$8.55	96	60,528	6,854	1435s
CC:Sonnet (+SAE)	$0.92	$0.92	33	8,747	1,462	391s
CC:Haiku	$2.88	$0.48	1,905	152,835	22,509	2271s
CC:Haiku (+SAE)	$0.48	$0.48	317	25,680	3,676	370s
Loopy:Sonnet	$2.72	$2.72	854,353	10,237	0	314s
Loopy:Haiku	$15.30	$2.55	18,336,594	158,691	0	2470s
Loopy:Qwen3	$5.01	$5.01	1,547,477	24,486	0	1421s

CC:Sonnet (hardened) delivers the best quality: T3=0.81, 100% pass rate, $1.39/item. Knowledge injection and two-phase execution add cost without measurable benefit on simple targets.

3.2 Finding 2: Model Is the Quality Gate, Framework Is the Cost Gate

Quality depends on model capability. Cost depends on framework engineering. These are **independent axes**.

Agent:Model	**Framework**	**T3**	**Cost**	**Input Tokens**	**Passed**
CC:Haiku	Claude Code	0.40	$0.45	300	No
Loopy:Haiku	Loopy (no compaction)	0.49	$2.55	18,336,594	No
CC:Sonnet	Claude Code	0.70	$0.92	—	Yes
Loopy:Sonnet	Loopy (with compaction)	0.73	$2.72	854,353	Yes

Table 4: gs-accessing-data-jpa: same item, same prompt across four agent/model combinations.

Read the rows: Haiku fails regardless of framework (T3 0.40–0.49, both fail jury). Sonnet passes regardless of framework (T3 0.70–0.73, both pass jury). The model determines *whether it works*. The framework determines *what it costs*.

Read the columns: Loopy:Haiku without compaction: 18M input tokens. Loopy:Sonnet with compaction (threshold 0.3): 854K input tokens — a **21**× reduction. Context compaction closes most of the framework gap, but Loopy:Sonnet still costs **3**× more than CC:Sonnet at API rates.

First Loopy:Sonnet run hit the $5 cost cap at turn 54 with compaction threshold 0.5. Lowering to 0.3 (compact at 60K tokens) let it complete at $2.72. Compaction tuning matters — too late and you burn budget on stale context.

3.3 Finding 3: There Is a Model Floor

Both CC:Haiku (T3=0.44) and Loopy:Haiku (T3=0.49) achieve decent **coverage** (71–94%) but fail the quality jury. Common failures: wrong test annotations (`@SpringBootTest` everywhere instead of `@WebMvcTest`), wrong injection patterns, no edge case tests. Below the model floor, no amount of framework engineering, knowledge injection, or prompt tuning helps. Haiku and Qwen3 are below that floor for Spring testing.

3.4 Finding 4: Knowledge Amplifies Capability

We built a lightweight **Stable Analysis Envelope (SAE)** — a pre-computed project report generated after compilation that gives the agent: dependency inventory, component classification (controllers, services, repos, entities), and **copy-paste import blocks** with full package names for each test slice.

We iterated the SAE through two versions:

SAE v1 (short annotations)
Component roles and dependency list only. Tells the agent *what exists* but not *how to import it*. Haiku + SAE v1 still launched 8 `jar tf` commands to discover package names.

SAE v2 (full import blocks)
Adds ready-to-paste import statements for every test slice (`@WebMvcTest`, `@DataJpaTest`, etc.). Haiku + SAE v2: zero jar exploration. Sonnet + SAE v2: still explored JARs (19 calls) but used the SAE to write higher-quality tests.

Model	SAE	T3	Cov%	Cost	jar-inspect	Passed
Haiku	none	0.43	71.4	$0.44	0	No
Haiku	v1	0.45	71.4	$0.39	8	No
Haiku	v2	0.35	71.4	$0.48	0	No
Sonnet	none	0.81	85.7	$0.83	18	Yes
Sonnet	v2	0.93	71.4	$0.92	19	Yes

Table 5: SAE ablation on gs-rest-service (single item, N=1). Rows are grouped by model. Sonnet + SAE v2 still explored JARs but wrote the best tests in the experiment (T3=0.93). Haiku + SAE v2 had zero jar exploration but wrote failing tests (T3=0.35).

Three things stand out:

1. **SAE eliminates jar exploration for Haiku** but not for Sonnet. Haiku follows the SAE literally; Sonnet explores beyond it. The behavioral change is model-dependent.
2. **SAE amplifies capable models.** Sonnet + SAE: T3 jumps from 0.81 to 0.93. Same model, same prompt, just better context.
3. **SAE cannot lift weak models.** Haiku + SAE v2: T3 *drops* to 0.35. Zero wasted exploration, but the tests it wrote were wrong.

Implication: the SAE is a quality lever for capable models and an efficiency lever for weaker ones — but it cannot substitute for model capability. Knowledge amplifies capability; it cannot create it.

3.5 Finding 5: Local Models Are Not Ready

Loopy:Qwen3 (Qwen3-Coder 30B via LM Studio) scored T3=0.30 with 0% pass rate. Even with 300 turns and 45-minute timeout, it couldn't write a single working test for the simplest project. It spent turns reading files and writing todo lists instead of writing tests.

3.6 Finding 6: Partial Knowledge Costs More Than Full or None

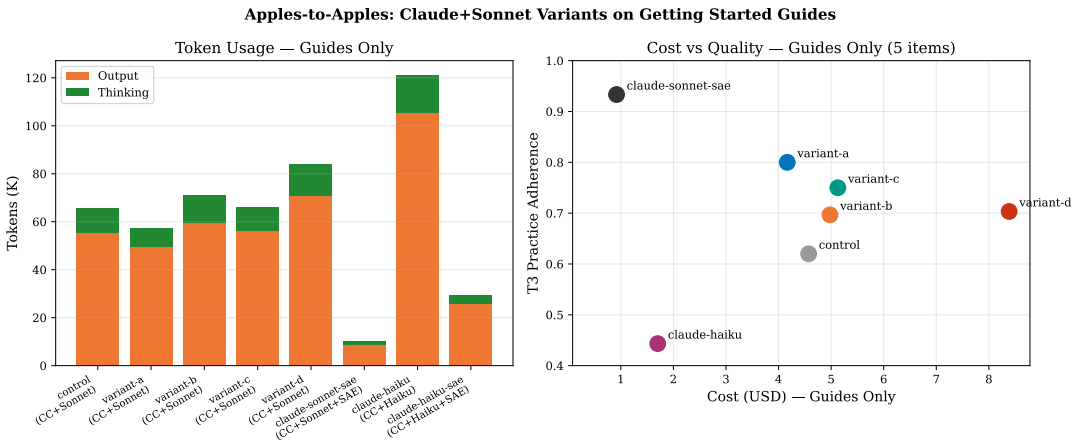

Figure 2: Guides-only comparison (apples-to-apples, 5 items). CC:Sonnet (+3 KB) uses more tokens than (+full KB) — partial knowledge triggers exploratory `jar tf` inspection of Spring Boot JARs to fill gaps.

3.7 Finding 7: Two-Phase Costs 2× for No Gain

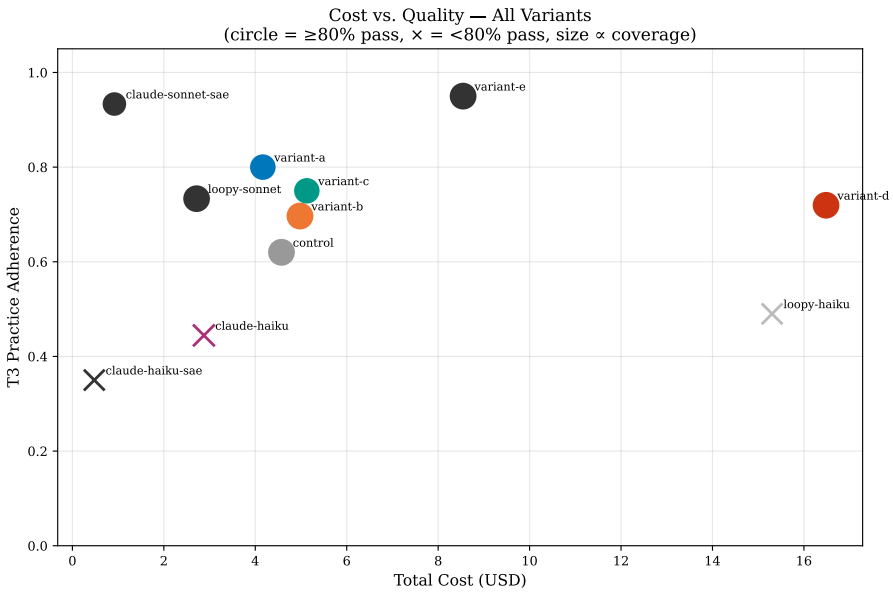

Figure 3: Cost vs. quality across all variants. CC:Sonnet (+SAE) is the new efficient frontier (T3=0.93, \$0.92). $\circ$ = $\geq$80% pass, $\times$ = <80% pass, size $\propto$ coverage.

4 Agent Behavior

4.1 Agent Internal Reasoning (Thinking Blocks)

The Agent Journal records more than tool calls. Each phase captures the full Claude response stream including *thinking blocks* — the model's internal chain-of-thought before each tool invocation. Until now the analysis pipeline stopped at tool calls. This section opens the layer above: what was the agent reasoning about?

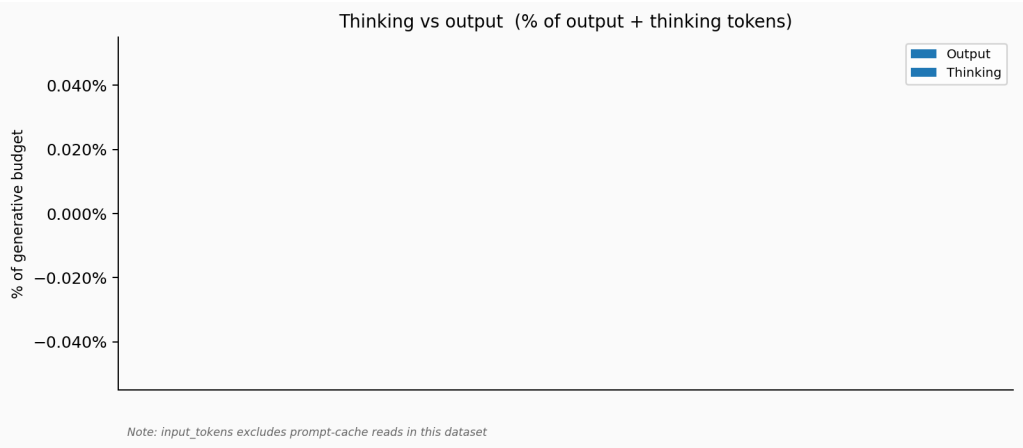

Figure 4: Thinking tokens as % of generative budget (output + thinking). Input tokens are excluded — prompt-cache reads were not captured in this dataset; `input_tokens` reflects only bare non-cached context (≈14 tokens per turn). The `+plan-act` variant allocates the highest share to thinking, consistent with its two-phase deliberation structure.

Q1: Where does the generative budget go?

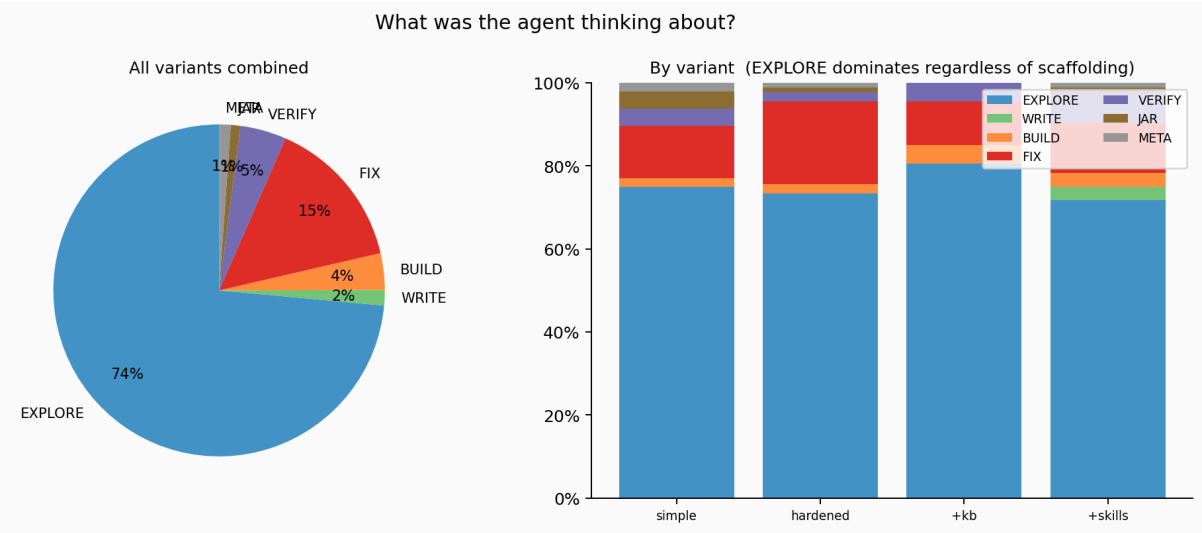

Figure 5: Keyword-classified thinking blocks (533 blocks, 7 variants). **EXPLORE dominates at 74% across all variants.** Stacked bars (right) show per-variant breakdown.

Q2: What was the agent thinking about? The dominant category is **EXPLORE** (74%) — understanding file structure, checking dependencies, orienting before acting. FIX (15%) covers error diagnosis. Everything else — BUILD, VERIFY, WRITE, JAR, META — accounts for the remaining 11%.

Key finding: exploration dominates even when knowledge scaffolding eliminates the corresponding *actions*. The `+skills` variant removes all `jar tf` inspection calls (zero JAR tool invocations vs. 18 for the hardened baseline), yet its thinking-block distribution still shows ≈74% EXPLORE. Skills change what the agent *does*; they do not change how it *reasons before acting*. The orientation instinct is model-level, not task-level.

This finding validates the Markov model from the opposite direction: if the agent is predominantly in an orientation mental state before acting, the tool-call transition matrix faithfully reflects the underlying intent distribution.

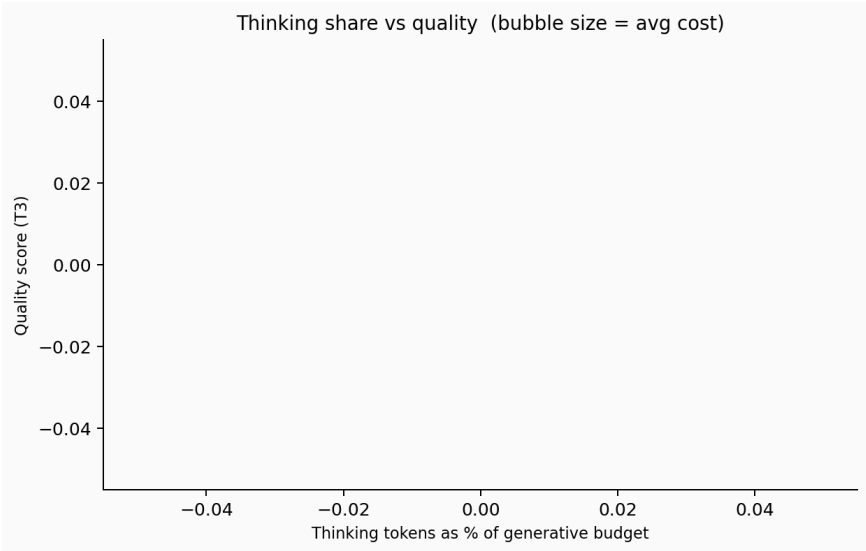

Figure 6: Thinking share of generative budget vs. T3 quality score. Bubble size $\propto$ average cost per item. Directional trend: higher thinking share associates with higher quality, but the relationship is confounded by variant complexity.

Q3: Does more thinking correlate with better quality?

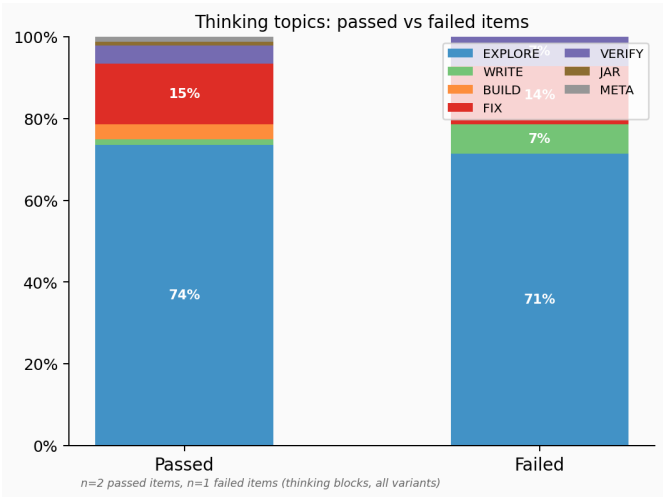

Figure 7: Thinking topic distribution: passed vs. failed items. FIX thinking is elevated in failing runs; EXPLORE share is slightly lower in passing runs, suggesting efficient orientation in successful attempts. (Pilot signal — 2 items in current dataset.)

Q4: Do passing and failing items think differently?

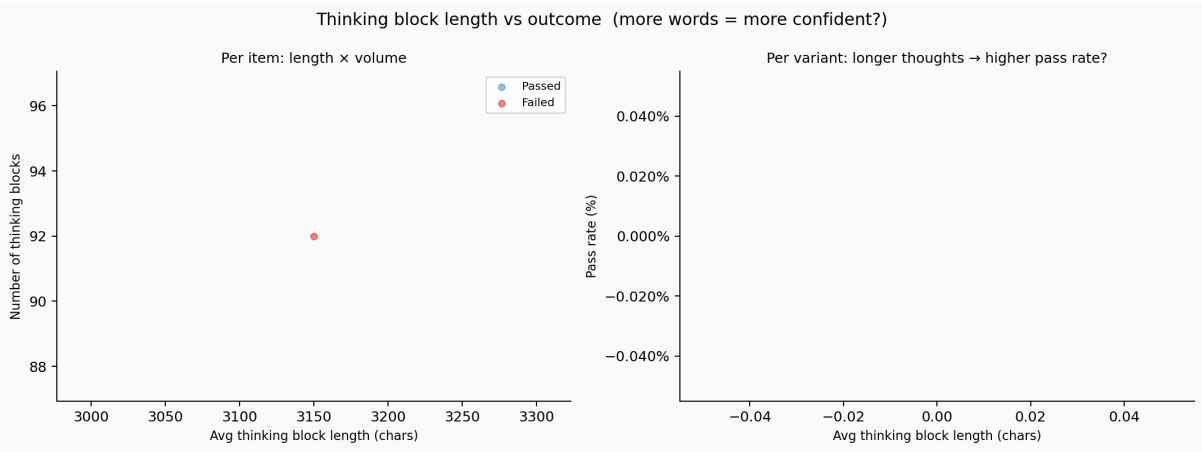

Figure 8: Left: per-item scatter of average thinking block length vs. block count, colored by pass/fail. Right: per-variant average block length vs. pass rate. Directional: longer thinking blocks associate loosely with better outcomes, but the signal is weak in this dataset.

Q5: More words = more confident?

Forward pointer: intent-state Markov analysis. The five charts above are descriptive. The deeper question is whether we can build $P(\text{action}_t \mid \text{intent}_t)$ — the agent's policy function mapping internal reasoning state to tool-call action. This would let us say: "knowledge scaffolding changes the agent's *policy function*, not just its behavior." Stable estimates require ∼300+ transitions per (intent_state, variant) cell; current data (2 items) is a pilot. This is the primary analytical milestone for the N=3 sweep.

4.2 Token Consumption

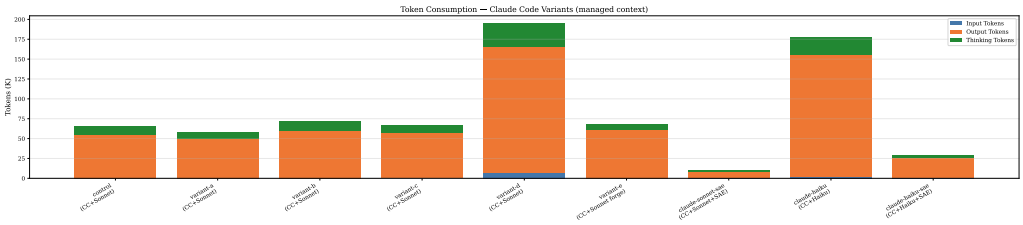

Figure 9: Token consumption — Claude Code variants. Context is managed by the CLI (compaction, summarization). Two-phase variant has the highest thinking tokens.

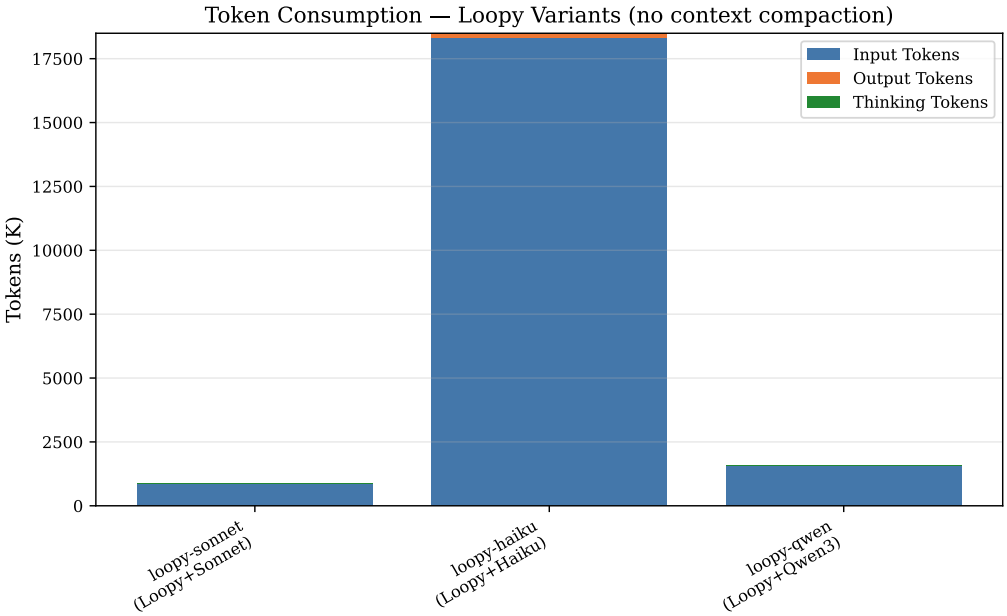

Figure 10: Token consumption — Loopy variants (note different Y scale). Without context compaction, input tokens grow monotonically — every prior tool result is re-sent on each turn. This is the dominant cost driver.

4.3 Tool Usage Patterns

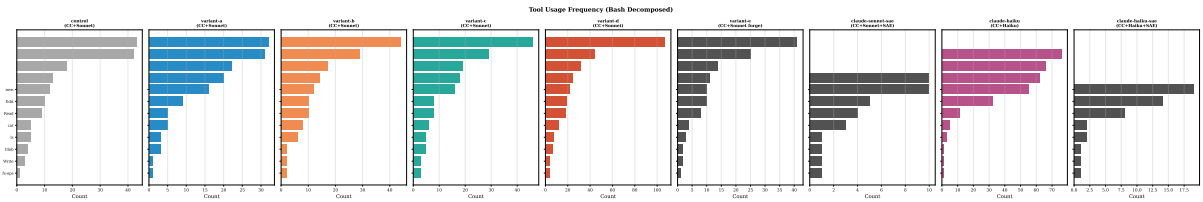

Figure 11: Tool call frequency per variant (bash decomposed to actual utilities). `jar-inspect` = `jar tf` commands probing Spring Boot JARs in `~/.m2` to discover test annotations. Knowledge-injected variants need fewer jar inspections.

4.4 Execution Timelines

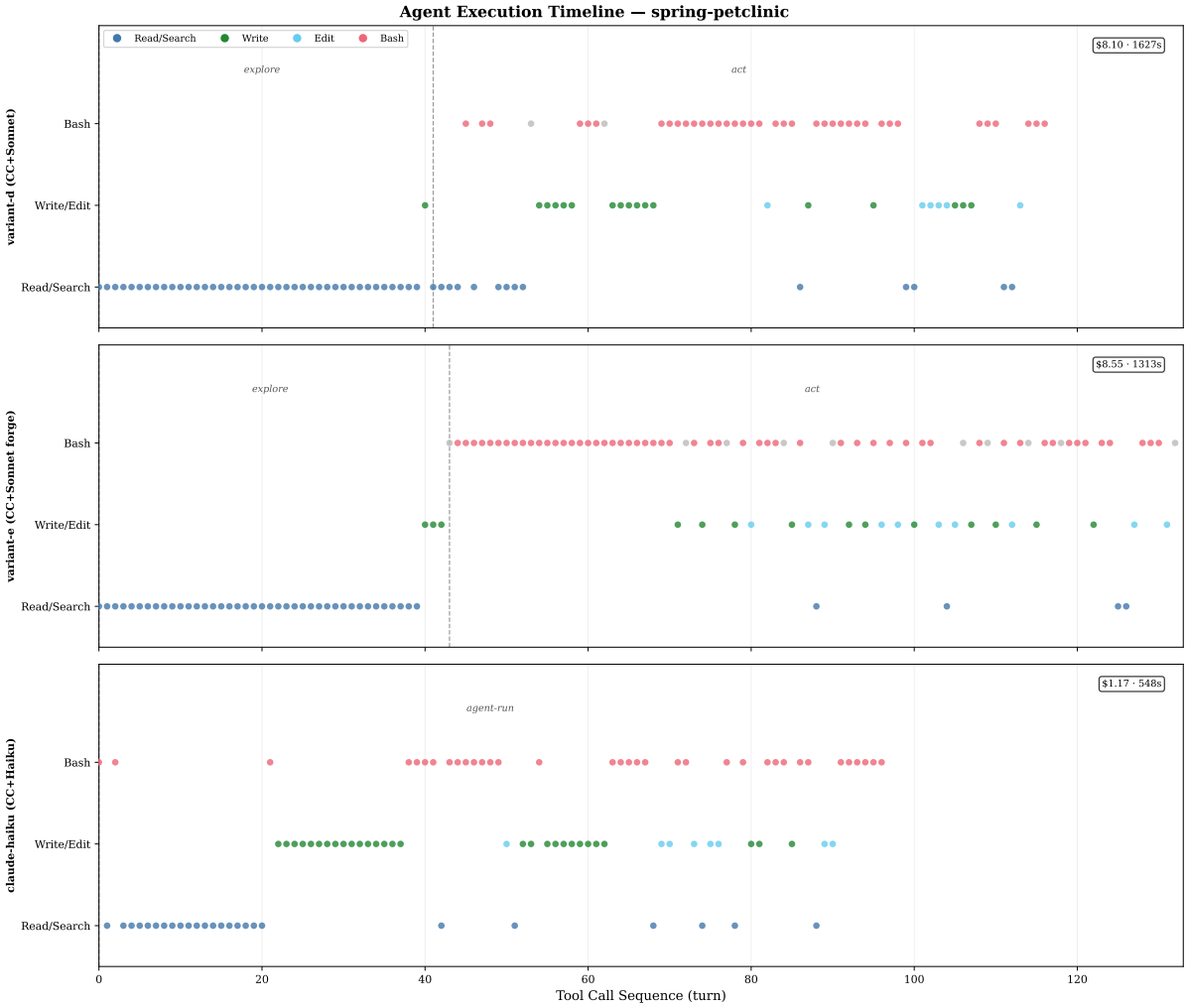

Figure 12: Agent execution timeline — Pet Clinic. Each dot is a tool call. Shared X axis shows relative effort across variants.

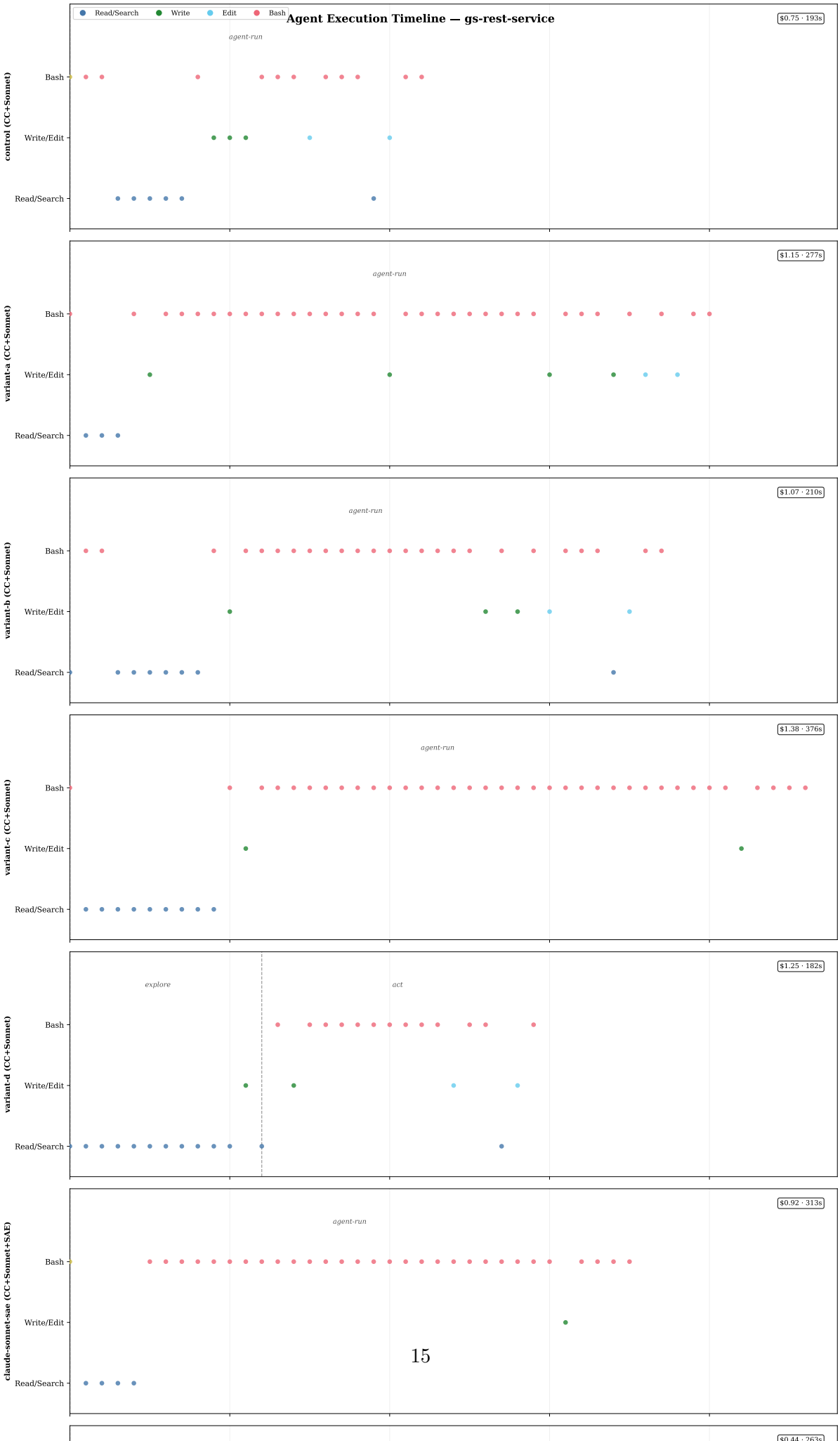
Agent Execution Timeline — gs-rest-service
Read/Search
Write
Edit
Bash
control (CC+Sonnet)
$0.75 · 193s
agent-run
variant-a (CC+Sonnet)
$1.15 · 277s
agent-run
variant-b (CC+Sonnet)
$1.07 · 210s
agent-run
variant-c (CC+Sonnet)
$1.38 · 376s
agent-run
variant-d (CC+Sonnet)
$1.25 · 182s
explore
act
claude-sonnet-sae (CC+Sonnet+SAE)
$0.92 · 313s
agent-run
$0.44 · 263s
Bash
Write/Edit
Read/Search

Table 6: T3 practice adherence per item (higher is better)

Item	CC:Sonnet (naive)	CC:Sonnet (hardened)	CC:Sonnet (+3 KB)	CC:Sonnet (+fu
gs-accessing-data-jpa	0.60	0.70	0.65	0.68
gs-messaging-stomp-websocket	0.50	0.60	0.50	0.50
gs-reactive-rest-service	0.50	0.93	0.70	0.90
gs-rest-service	0.67	0.88	0.85	0.88
gs-securing-web	0.83	0.88	0.78	0.78
spring-petclinic	—	—	—	—

Table 7: Efficiency breakdown per variant (higher is better, except tokens/duration)

Agent:Model	Build Err	Cost Eff	Recovery	Composite	Avg Out Tok	Avg Think Tok	Avg Durat
CC:Sonnet (naive)	0.9	0.82	0.9	0.88	11,086	2,032	221s
CC:Sonnet (hardened)	0.97	0.83	0.97	0.94	9,869	1,558	216s
CC:Sonnet (+3 KB)	0.85	0.8	0.85	0.84	11,894	2,266	237s
CC:Sonnet (+full KB)	0.88	0.79	0.88	0.85	11,203	1,985	334s
CC:Sonnet (two-phase)	0.65	0.55	0.75	0.65	26,436	4,866	508s
CC:Sonnet (forge)	0	0	0.62	0.17	60,528	6,854	1313s
CC:Sonnet (+SAE)	0.88	0.82	0.88	0.86	8,747	1,462	313s
CC:Haiku	0.67	0.9	0.81	0.77	25,472	3,752	294s
CC:Haiku (+SAE)	0	0.9	0.62	0.41	25,680	3,676	332s
Loopy:Sonnet	1	0.46	1	0.86	10,237	0	227s
Loopy:Haiku	1	0.49	1	0.86	26,448	0	330s
Loopy:Qwen3	1	0	1	0.73	24,486	0	1419s

5 Per-Item Breakdown

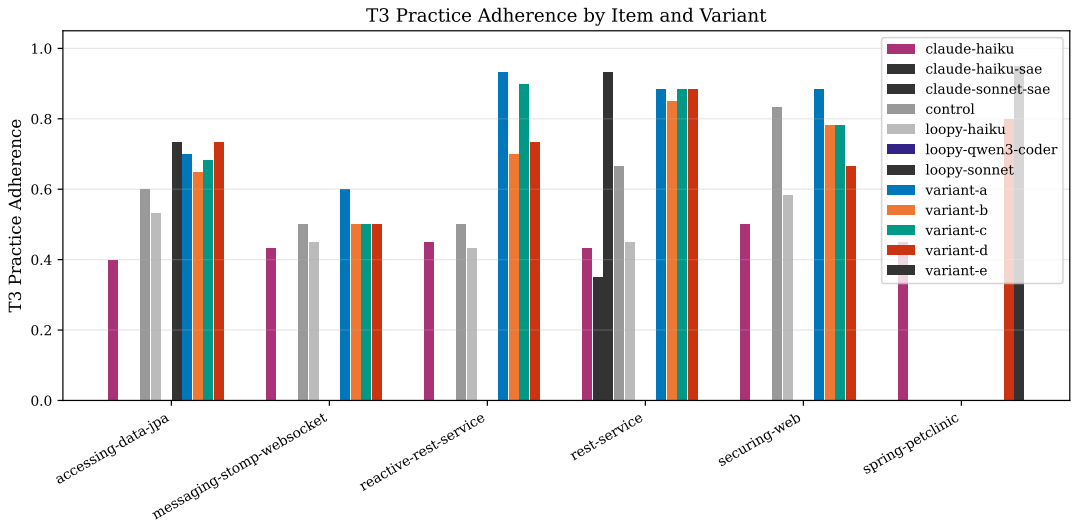

Figure 14: T3 practice adherence by item and variant.

6 Efficiency

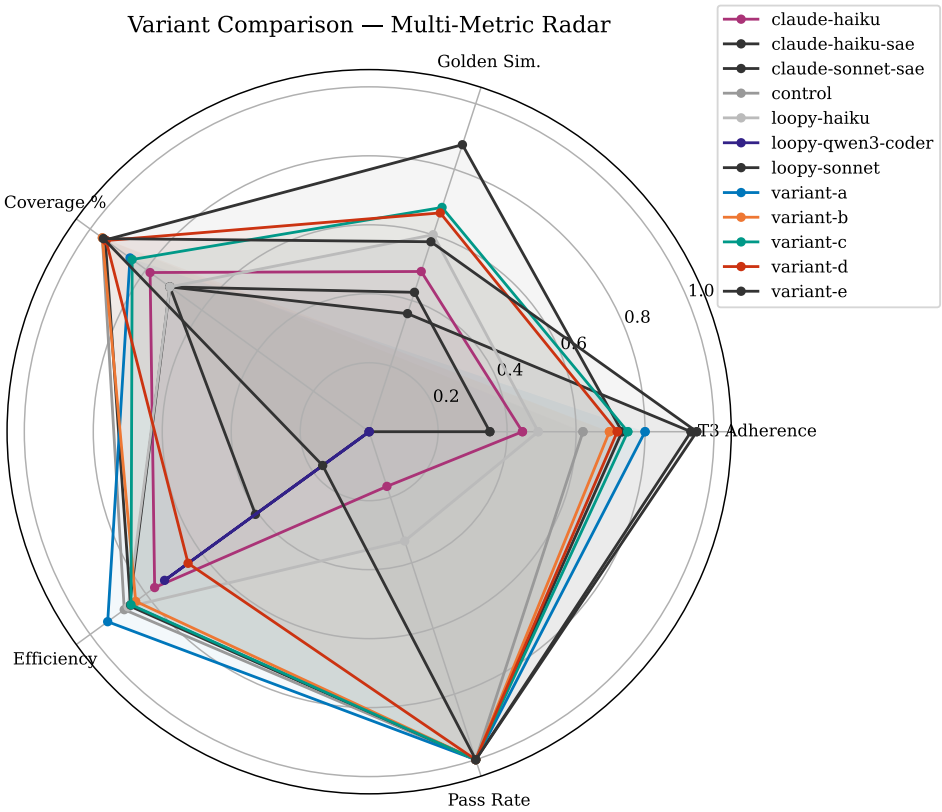

Figure 15: Multi-metric radar chart across all variants.

7 Discussion

The central thesis is:

$$\textit{knowledge} + \textit{structured execution} > \textit{model alone}$$

The simple Spring guides provide the **null condition**: when the task is trivial, the model alone (with a good prompt) is sufficient. Knowledge injection adds cost without measurable benefit. The two-phase pattern adds both cost and complexity.

The Pet Clinic results validate the thesis from the positive direction. The plan/act two-phase variant achieves T3=0.95 on a complex, multi-layer application — the highest quality score in the experiment. Structured execution pays off when the task exceeds what the model can solve in a single pass.

The SAE finding adds a nuance: knowledge amplifies capable models (Sonnet + SAE: T3=0.93) but cannot lift weak ones (Haiku + SAE: T3=0.35). This means the thesis should be read as **knowledge investment is justified when (a) the model is above the capability floor and (b) task complexity exceeds what the model can infer from context alone**. Below the model floor, no amount of knowledge, tooling, or structured execution helps.

The cross-model and cross-framework comparisons confirm that model capability and framework engineering are independent axes. The model determines *whether it works*; the framework determines *what it costs*. Loopy:Sonnet matches CC:Sonnet on quality but costs 3× more. Context compaction (21× token reduction) closes most of the gap, suggesting that framework cost is an engineering problem, not a fundamental one.

7.1 Limitations

- **N=1 per cell**: Every score is a single observation. Differences of ∼0.05 are indistinguishable from stochastic variation.
- **Simple dataset dominates**: 5 of 6 items are single-layer, 1–3 class projects. Pet Clinic is the only complex target.
- **Multiple models tested, but coverage is uneven**: Sonnet was tested across all knowledge treatments; Haiku and Qwen3 only on a subset. Cross-model comparisons are directional, not controlled.
- **No repeated trials**: Stochastic variation in LLM outputs means a single run may not represent typical behavior.
- **T3 is LLM-judged**: The practice adherence score comes from an LLM, which introduces its own noise. Golden similarity (AST-based) is deterministic but measures structure, not behavior.
- **Anthropic models only for agent**: All Claude Code variants use Anthropic models. Qwen3 was tested only through Loopy. Direct comparison of local vs. API models on the same framework is not yet available.

8 Conclusion

1. **Prompt engineering is the biggest lever — and the cheapest.** T3: 0.62 → 0.81. +29% quality, same model.
2. **Coverage lies.** 94% coverage, T3=0.45. Without the jury you'd ship bad tests and never know.
3. **There is a model floor.** Below it, no amount of tooling, knowledge, or prompting helps. Haiku, Qwen3 — can't do it.
4. **Model is the quality gate; framework is the cost gate.** Loopy:Sonnet (T3=0.73) ≈ CC:Sonnet (T3=0.70) — same quality, 3× cost. The model determines *whether it works*; the framework determines *what it costs*.
5. **Context compaction is the hidden cost multiplier.** Loopy without compaction: 18M input tokens. With compaction (threshold 0.3): 854K. 21× reduction. Compaction tuning matters.
6. **Knowledge amplifies capability — it cannot create it.** Sonnet + SAE: T3=0.93. Haiku + SAE: T3=0.35. Same pre-computed analysis, opposite results.
7. **Partial knowledge is worse than none.** Giving 3 files triggers more exploration than giving the full KB or nothing at all.
8. **Structured execution shines on hard tasks.** Pet Clinic with plan/act: T3=0.95. Two-phase adds 2× cost on simple targets but pays off on complex ones.

The experiment infrastructure — automated four-tier judge cascade, golden test AST comparison, efficiency tracking, agent journal capture with thinking-block analysis, and Markov chain behavioral modeling — is validated and reusable. The dataset, prompts, knowledge base, judge rubrics, and all raw results are published for reproducibility. Next steps: larger N for statistical power, harder targets (multi-module projects, legacy codebases), and cross-model controlled comparisons where the same knowledge treatments are applied to multiple models on the same framework.